Mongo Shell Delete User And Profile Guide

Use this guide when you want to delete one user and that user's profile directly from mongosh. The script is split into a preview step and a delete step. Always run preview first.

> Safety rule: edit only one identifier at the top. Do not set email, mobile, profileNumber, userId, and profileId all at once unless you are intentionally cross-checking the same person.

1. Open Mongosh

Connect to the correct database before running the code. Use the same database that your server MONGO_URI points to.

```
// Example only. Use your real connection string or open mongosh from MongoDB Compass.
mongosh "mongodb+srv://USER:PASSWORD@HOST/nichayavedika"

// Check current database before deleting anything.
db.getName()
```

> Comment: If db.getName() is not the expected production or test database, stop and reconnect to the correct database.

2. Preview Script

Paste this first. It finds the user/profile and shows counts for related records. It does not delete anything.

```
// Pick one identifier. Leave the others as empty strings.
const email = "user@example.com";       // Finds by users.email and matching OTP/activity email.
const mobile = "";                      // Finds by users.mobile and matching OTP mobile.
const profileNumber = "";               // Finds by profiles.profileNumber, such as PN000001.
const userId = "";                      // Finds by users._id. Must be a Mongo ObjectId string.
const profileId = "";                   // Finds by profiles._id. Must be a Mongo ObjectId string.

// Build queries from the identifier you filled in above.
const userQuery = {};
const profileQuery = {};

if (email) userQuery.email = email.toLowerCase();
if (mobile) userQuery.mobile = mobile;
if (userId) userQuery._id = ObjectId(userId);

if (profileId) profileQuery._id = ObjectId(profileId);
if (profileNumber) profileQuery.profileNumber = profileNumber;

// Find user/profile. If only one side is found, find the linked other side.
let user = Object.keys(userQuery).length ? db.users.findOne(userQuery) : null;
let profile = Object.keys(profileQuery).length ? db.profiles.findOne(profileQuery) : null;

if (!profile && user) {
  profile = db.profiles.findOne({ user: user._id });
}

if (!user && profile && profile.user) {
  user = db.users.findOne({ _id: profile.user });
}

const uid = user?._id;
const pid = profile?._id;

print("User:");
printjson(user);

print("Profile:");
printjson(profile);

// Related records that should be cleaned with the user/profile.
const interestFilter = {
  $or: [
```

```
    ...(uid ? [{ sender: uid }, { fromUser: uid }] : []),
    ...(pid ? [{ receiverProfile: pid }, { toProfile: pid }] : [])
  ]
};

const otpFilter = {
  $or: [
    ...(email ? [{ email: email.toLowerCase() }] : []),
    ...(mobile ? [{ mobile }] : [])
  ]
};

const activityFilter = {
  $or: [
    ...(uid ? [{ user: uid }] : []),
    ...(email ? [{ userEmail: email.toLowerCase() }] : [])
  ]
};

print("Preview counts:");
printjson({
  users: uid ? db.users.countDocuments({ _id: uid }) : 0,
  profiles: pid ? db.profiles.countDocuments({ _id: pid }) : 0,
  interests: interestFilter.$or.length ? db.interests.countDocuments(interestFilter) : 0,
  payments: uid ? db.payments.countDocuments({ user: uid }) : 0,
  otps: otpFilter.$or.length ? db.otps.countDocuments(otpFilter) : 0,
  activityLogs: activityFilter.$or.length ? db.activitylogs.countDocuments(activityFilter) : 0,
  adminAuditLogs: uid ? db.adminauditlogs.countDocuments({ targetUser: uid }) : 0
});
```

Preview Comments

Line / area	What it does
Identifier block	You edit one value: email, mobile, profileNumber, userId, or profileId.
userQuery/profileQuery	Builds the Mongo query safely from the identifier you chose.
Find user/profile	Finds both linked records even if you only gave one side.
interestFilter	Catches interests sent by the user and interests received by the profile.
otpFilter	Finds OTP rows tied to the user email/mobile.
activityFilter	Finds activity logs tied to the user id or email.
Preview counts	Shows exactly how many records would be removed in each collection.

If users or profiles shows 0, the identifier did not match. Do not run delete until the preview shows the exact record you intend to remove.

3. Delete Script

Run this only after the preview looks correct. It blocks admin, super_admin, and oper_admin users by default.

```
if (!uid && !pid) {
  throw new Error("No matching user/profile found. Nothing deleted.");
}

// Safety guard: do not delete admin accounts through this script.
if (user && ["admin", "super_admin", "oper_admin"].includes(user.role)) {
  throw new Error("Refusing to delete admin user from mongosh script.");
}

// Delete related records first, then profile, then user.
printjson(interestFilter.$or.length ? db.interests.deleteMany(interestFilter) : { deletedCount: 0 });
printjson(uid ? db.payments.deleteMany({ user: uid }) : { deletedCount: 0 });
printjson(otpFilter.$or.length ? db.otps.deleteMany(otpFilter) : { deletedCount: 0 });
printjson(activityFilter.$or.length ? db.activitylogs.deleteMany(activityFilter) : { deletedCount: 0 });
printjson(uid ? db.adminauditlogs.deleteMany({ targetUser: uid }) : { deletedCount: 0 });
printjson(pid ? db.profiles.deleteOne({ _id: pid }) : { deletedCount: 0 });
printjson(uid ? db.users.deleteOne({ _id: uid }) : { deletedCount: 0 });

print("Delete complete.");
```

Delete Comments

Command	What it deletes
deleteMany(interests)	Interests sent by the user or received by the profile.
deleteMany(payments)	Payment records for the user.
deleteMany(otps)	OTP records for the chosen email/mobile.
deleteMany(activitylogs)	Activity logs tied to user id or user email.
deleteMany(adminauditlogs)	Admin audit rows where this user is the target.
deleteOne(profiles)	The matched profile document.
deleteOne(users)	The matched user account.

4. After Delete Verification

Run the preview script again with the same identifier. The matching user/profile should be null and the related counts should be 0.

```
// Re-run the preview block after delete.
// Expected result: user/profile not found and related counts are 0.
```

> Tip: If activity log count remains because old records use a different email spelling, search manually with db.activitylogs.find({ userEmail: /email/i }).limit(5).

5. Safer Alternative

The project also has a Node/Mongoose script that does the same cleanup with a built-in dry run:

```
cd C:\Projects\nichayavedika_v2\server
npm.cmd run delete:user -- --email user@example.com
npm.cmd run delete:user -- --email user@example.com --confirm
```

> Use the Node script when possible. Use mongosh only when you are already connected to the database and need direct database control.